Lifecycle və ASP.NET Core sorğusunun ömrü

Lifecycle bir varlığın yaranmasından işinin bitməsinə qədər keçdiyi mərhələlərdir. .NET müsahibəsində bu söz tək bir mövzunu bildirmir. Obyektin yaddaşdakı ömrü, DI xidmətinin paylaşılma müddəti, HTTP sorğusunun emalı və tətbiqin başlama dayanma mərhələləri ayrı anlayışlardır. Bu dərs onları bir sifariş nümunəsi üzərində əlaqələndirir.

Bu sənəd `DependencyInjection.pdf`, `di_practical.pdf`, `files/02_aspnet_core.pdf` və CLR materiallarını tamamlayır. Xüsusilə “scoped hər connection üçün yaranır”, “singleton mütləq startup zamanı yaranır” və “Dispose obyektin yaddaşını silir” kimi yanlış nəticələrdən qoruyur.

Mündəricat

Lifecycle və ASP.NET Core sorğusunun ömrü **1**

1 Başlamazdan əvvəl əsas anlayışlar . . . 2

2 Dörd müxtəlif ömür . . . 2

3 Sorğu addım addım haradan keçir . . . 2

4 Middleware-in əvvəl və sonra davranışı . . . 3

5 Obyektin ömrü və Dispose fərqi . . . 4

6 Tətbiqin başlanması və dayanması . . . 4

7 Yoxlama sualları və cavablar . . . 4

8 Rəsmi mənbələr . . . 5

1 Başlamazdan əvvəl əsas anlayışlar

Proses əməliyyat sisteminin işləyən proqram üçün ayırdığı mühitdir. Prosesin yaddaşı və resursları olur. **Thread** proses daxilində kodu icra edən axındır. Eyni prosesdə bir neçə thread eyni obyektlərə çata bilər. **Server** burada şəbəkədən gələn sorğulara cavab verən tətbiqdir. **HTTP** müştəri ilə server arasında sorğu və cavabların qaydasını müəyyən edən protokoldur. Müştəri brauzer, mobil proqram və ya başqa server ola bilər.

Request bir HTTP sorğusudur: məsələn, `GET /orders/42`. `GET` məlumat oxumaq niyyətini, `/orders/42` isə ünvanı göstərir. **Response** serverin cavabıdır. Status kodu nəticənin kateqoriyasını bildirir: `200` uğurlu, `404` tapılmadı, `500` server xətası. **Endpoint** konkret sorğunun çatdığı işləyici kod və onu seçən qaydalardır. **Instance** sinifdən yaradılmış konkret obyekt nüsxəsidir.

DI asılılıqların obyektə xaricdən verilməsidir. **Container** hansı xidmətin necə yaradılacağını bilən mexanizmdir. **Scope** xidmət nüsxələrinin paylaşılması və təmizlənməsi üçün sərhəddir. Scope bir C# `{ }` bloku ilə eyni anlayış deyil, ayrıca yaradılan DI obyektidir. HTTP sorğusu üçün framework adətən bir scope yaradır.

2 Dörd müxtəlif ömür

Ömür	Başlanğıc və son
Tətbiq	Host qurulur, xidmət etməyə başlayır, dayanma siqnalı alır və bağlanır. Host tətbiqin işləməsini idarə edən çərçivədir.
HTTP sorğusu	Server sorğunu qəbul edir, emal zəncirindən keçirir, cavabı tamamlayır.
DI xidməti	Container xidməti yaradır və lifetime qaydasına uyğun paylaşır; sahibi bitəndə resurslarını təmizləyir.
Yaddaşdakı obyekt	Yaddaş ayrılır, konstruktor işləyir, istinadlarla istifadə olunur; əlçatmaz olduqdan sonra GC yaddaşı sonradan geri ala bilər.

Bir sorğu bitə bilər, amma singleton yaşamağa davam edər. Bir obyekt `Dispose` oluna bilər, amma ona istinad qalırsa yaddaşda qalar. Eyni TCP bağlantısı ilə bir neçə HTTP sorğusu gedə bilər; buna görə scope-u bağlantı ilə eyniləşdirmə. TCP şəbəkədə baytların etibarlı çatdırılmasını təmin edən nəqliyyat protokoludur.

3 Sorğu addım addım haradan keçir

Müştəri `GET /orders/42` göndərir. Ünvanın IP-yə çevrilməsi DNS-in işidir; IP şəbəkədə cihazın ünvanıdır. HTTPS istifadə olunursa TLS bağlantının şifrələnməsini və server kimliyinin yoxlanmasını təmin edir. Bu şəbəkə mərhələləri hər sorğuda sıfırdan təkrarlanmaya bilər, çünki mövcud bağlantı təkrar istifadə oluna bilər.

Kestrel ASP.NET Core-un HTTP serveridir. Qarşısında reverse proxy ola bilər: məsələn, daxil olan trafiki tətbiqə ötürən server. Kestrel sorğu üçün `HttpContext` yaradır. Bu obyekt sorğu ünvanını, başlıqları, cavabı, istifadəçi məlumatını və ləğv siqnalını bir yerdə saxlayır. Header, yəni başlıq, sorğu və cavabla gedən əlavə məlumatdır; məsələn, məzmunun formatı.

Sonra sorğu **middleware pipeline** daxilində irəliləyir. Middleware sorğunu qəbul edən, növbəti mərhələni çağıra və cavab qayıdanda əlavə iş görə bilən komponentdir. Pipeline bu komponentlərin ardıcıllığıdır. Xəta tutma, təhlükəsiz ünvana yönləndirmə, routing, kimlik yoxlama və icazə yoxlama bu zəncirə daxil ola bilər.

Routing ünvan və HTTP metoduna əsasən endpoint seçir. **Authentication** “kimdir?” sualına cavab verir. **Authorization** “bu əməliyyatı edə bilərmi?” sualını yoxlayır. Kimliyi məlum istifadəçinin də bütün sifarişlərə giriş icazəsi olmaya bilər. `401` adətən etibarlı kimlik məlumatının tələb olunduğunu, `403` isə icazənin verilmədiyini bildirir.

MVC controller işlənirsə, model binding sorğudakı məlumatı metod parametrlərinə çevirir. Validation həmin məlumatın qaydalara uyğunluğunu yoxlayır. Controller HTTP ilə iş məntiqi arasında əlaqə qurur. Action onun konkret sorğunu işləyən metodudur. Filter isə MVC action və nəticə emalının müəyyən mərhələlərinə qoşulan mexanizmdir. Minimal API istifadə olunursa ayrıca controller və MVC filter mərhələsi olmaya bilər.

Controller xidmət çağırır, xidmət lazım gəlsə verilənlər bazası ilə işləyir. **DbContext** EF Core-da sorğuların göndərilməsi və obyekt dəyişikliklərinin izlənməsi üçün istifadə olunan iş vahididir. EF Core C# obyektlərini cədvəl məlumatları ilə əlaqələndirən kitabxanadır. Nəticə DTO-ya çevrilə bilər: DTO şəbəkə ilə göndərmək üçün seçilmiş məlumatları saxlayan sadə obyektdir. Serialization obyektin JSON kimi ötürülən formata çevrilməsidir. JSON açar və dəyərlərlə mətn şəklində məlumat formatıdır.

Nəhayət, cavab geri qayıdır və scope təmizlənir. Sorğu ləğv ediləndə `RequestAborted` token-i bunu bildirir. Token əməliyyata “dayanmaq istənilir” siqnalını ötürür; kod və sürücü onu nəzərə almalıdır. Cavab yazılmağa başladıqdan sonra status və başlıqların dəyişdirilməsi məhdudlaşır.

4 Middleware-in əvvəl və sonra davranışı

Aşağıdakı C# fraqmenti `Program.cs` daxilində `app.Run()` çağırışından əvvəl yerləşdirilir. `app` qurulmuş `WebApplication` obyektidir. `Console.WriteLine` ekrana qeyd yazır. Lambda `=>` işarəsi ilə verilən qısa funksiya ifadəsidir.

C#

```
app.Use(async (context, next) =>
{
    Console.WriteLine("A evvel");
    await next(context);
    Console.WriteLine("A sonra");
});

app.Use(async (context, next) =>
{
    Console.WriteLine("B evvel");
    await next(context);
    Console.WriteLine("B sonra");
});

app.MapGet("/hello", () => "Salam");
```

Uğurlu `/hello` sorğusunda sıra belədir: A əvvəl, B əvvəl, endpoint, B sonra, A sonra. `next` növbəti komponenti təmsil edir. `await` həmin asinxron əməliyyatın tamamlanmasını bloklamadan gözləyir. Növbəti hissə xəta atarsa “sonra” sətri avtomatik işləməyə bilər; mütləq təmizlik üçün `try/finally` lazımdır. `finally` nəticədən asılı olmayaraq icra edilən təmizlik blokudur.

Middleware `next` çağırmadan cavab qaytarırsa zənciri qısaldır. Məsələn, icazəsiz sorğuya cavab verən mərhələ bazaya müraciəti dayandıra bilər. Sıra buna görə əhəmiyyətlidir. Xəta tutan komponent erkən, authentication isə authorization-dan əvvəl yerləşdirilir. Konkret sıra istifadə olunan funksiyalara görə qurulur; bütün tətbiqlər üçün eyni kor siyahı tətbiq edilmir.

5 Obyektin ömrü və Dispose fərqi

Konstruktor obyekt yaradılarkən onun ilkin vəziyyətini quran metoddur. **İstinad** obyektə çatmağa imkan verən əlaqədir. **Managed heap** .NET-in obyektlər üçün idarə etdiyi yaddaş sahəsidir. **GC**, Garbage Collector, proqramın artıq çata bilmədiyi obyektlərin yaddaşını avtomatik geri alan mexanizmdir. Obyektin faydalı işi bitən kimi GC-nin işləməsi zəmanətli deyil.

Fayl və şəbəkə bağlantısı kimi resurslar vaxtında buraxılmalıdır. **IDisposable** bunun üçün `Dispose()` müqaviləsini, **IAsyncDisposable** isə asinxron `DisposeAsync()` müqaviləsini verir. `using` və `await using` müvafiq təmizlik metodunun çağırılmasını təşkil edir. Dispose yaddaşı dərhal boşaltmır və GC-ni məcbur etmir.

C#

```
using var file = File.OpenRead("data.txt");
// file-dan oxuma aparilir.
// Cari blok bitende Dispose cagrilir.
```

Bu fraqment üçün `System.IO` ad sahəsi lazımdır. Ad sahəsi əlaqəli tipləri qruplaşdıran addır. `var` dəyişənin tipinin sağdakı ifadədən kompilyator tərəfindən müəyyən edilməsidir. Kompilyator C# mətnini icra edilən formaya çevirən alətdir. `//` kodda işlədilməyən izah sətridir.

Container yaratdığı və təmizlənməsini izlədiyi disposable xidmətləri scope və ya provider bitəndə təmizləyir. Provider container-dən xidmət nüsxələrini əldə etməyə imkan verən obyektdir. Özün yaratdığın obyekti `AddSingleton(existingObject)` ilə verəndə onun təmizlənməsi sənin məsuliyyətindir. DI-dan aldığın `DbContext`-i onu alan hər servis ayrıca dispose etməməlidir; sahibi olan scope bunu etməlidir.

6 Tətbiqin başlanması və dayanması

Başlanğıcda konfiqurasiya oxunur, xidmətlər qeydiyyatdan keçir, host qurulur və server sorğuları dinləməyə başlayır. Konfiqurasiya bağlantı ünvanları kimi davranış parametrləridir. Qeydiyyat sinfin dərhal yaradılması demək deyil: singleton çox vaxt ilk dəfə tələb olunanda yaradılır.

Hosted service host tərəfindən başladılan və dayandırılan işdir. `BackgroundService` uzunmüddətli fon işi üçün baza sinifdir və adətən singleton kimi idarə edilir. Fon işində hər əməliyyat üçün ayrıca scope yaratmaq olar. Dayanma token-i gəldikdə dövr dayanmalı, resurslar buraxılmalıdır. **Graceful shutdown** davam edən işlərə məhdud vaxt ərzində səliqəli bitmək imkanı verən dayanmadır. Proses qəfil öldürülsə `finally` və Dispose-un işləməsi zəmanətli deyil; vacib işlər davamlı yaddaşda saxlanmalıdır.

HTTP sorğusunun içində başladılıb gözlənilməyən fon işi sorğunun scope-undan daha uzun yaşaya bilər. İş həmin `DbContext`-dən istifadə etdikdə artıq dispose olunmuş obyektlə qarşılaşar. Etibarlı fon emalı üçün ayrıca scope və lazım olduqda davamlı növbə istifadə edilir. Növbə işləri sonradan emal olunmaq üçün saxlayan mexanizmdir.

7 Yoxlama sualları və cavablar

Bir istifadəçi iki sorğu göndərsə scoped obyekt eynidirmi? Adətən xeyr. Scope istifadəçinin hesabına bağlı deyil, ayrı HTTP sorğularına bağlıdır.

Eyni sorğuda iki thread işləsə iki DbContext yaranırmı? Scope avtomatik thread-ə görə bölünmür. Eyni scoped context paylaşıla bilər və paralel istifadə təhlükəlidir. DbContext eyni anda bir neçə əməliyyatı təhlükəsiz icra etmək üçün hazırlanmayıb.

Dispose çağırılıbsa obyekt yox olurmu? Xeyr. Resurslar buraxılır, amma obyektə istinad qala bilər. Onu yenidən istifadə etmək xəta və ya yanlış davranış yarada bilər.

Singleton bütün serverlərdə birdirmi? Xeyr. Eyni qeydiyyat üçün bir root provider daxilində paylaşılır. İki prosesin və ya iki server nüsxəsinin ayrı singleton-ları var.

Məşq. A və B middleware-lərini əlavə et. B-də `next` çağırmadan cavab yaz. A-nın “sonra” hissəsi işləyəcək, endpoint işləməyəcək. Sonra B-də xəta yarat və A-nın `try/finally` ilə təmizlik etdiyini müşahidə et. Məqsəd giriş sırası ilə geri qayıtma sırasını ayırmaqdır.

8 Rəsmi mənbələr

- Microsoft Learn - ASP.NET Core middleware
- Microsoft Learn - DI service lifetimes
- Microsoft Learn - DbContext lifetime

Mənbələr 28 oktyabr 2026 tarixində yoxlanılıb. Kod hissələri tədris fraqmentləridir; tam layihə və bütün `using` sətirləri göstərilmir.